ELEC 271

Digital Systems

Fall, 2026

Instructor: Dr. Muhammad Saad Asad

Modified from notes provided by Prof. Ryan Grant

Copyright © Prof. Ryan Grant

Course Administration

- Instructors:
- Dr. Muhammad Saad Asad

  Email: asad.m@queensu.ca

  Office hours: by appointment

- TA (coordinators):
  - TBD

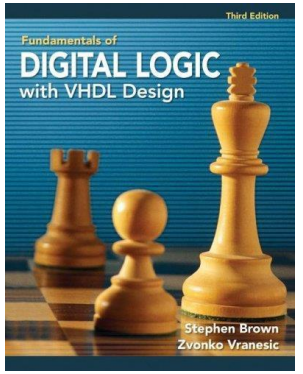
- Course Web site: **OnQ**
- Prerequisites: APSC-171, APSC-172, APSC-174
- Lectures & tutorials: See your schedule!
- Problem Sets: 7 sets; **no marking** – solutions will be posted after the tutorial

Course Administration (Cont'd)

- Midterm: **in class – October 29th – room assignments TBD**
  - Covers all material up to near the midterm. Information on exactly what is fair game for the midterm will be provided closer to the date.
- Lecture slides:
  - Available on OnQ as the term progresses. Slides are used to teach the fundamental concepts, and for quick reference to code, block diagrams, timing diagrams, etc.
  - There is a **significant amount of design material that will be covered in class but not necessarily included in the slides**. You're highly encouraged to come to class as much of the material in this course is best covered using in-person blackboard solutions.

Course Administration (Cont'd)

- Grading:
  - Midterm: 30%
  - Labs 30%
    (4 labs, no lab report submission, labs graded during lab session):
  - Final exam: 40%



- Textbook (Required): available in campus bookstore
  - **Fundamentals of Digital Logic with Verilog Design**

  ISBN: 9780073380544 Stephen Brown and Zvonko Vranesic, McGraw-Hill

Course Objectives

- Learn the basic principles and limitations behind digital logic systems
- Understand the current technology being used in this area
- Understand the fundamental electronic aspects of digital circuits
- Analyze, synthesize, and optimize multilevel digital systems
- Design arithmetic, combinational, and sequential digital systems
- Understand how digital circuits are designed in industry using hardware description languages (VHDL) and CAD tools
- Understand the real-world applications of what you are learning
- Develop the ability to creatively design new systems using your knowledge gained in this course

Course Outline

- This is a tentative course outline. Some topics and sections may be added or removed.
- Section 1: Design Concepts
  - Design process, logic circuits, binary numbers
- Section 2: Introduction to Logic Circuits
  - Logic gates, functions, truth tables, Boolean algebra, analysis and synthesis of simple circuits, sum-of-products, product-of-sums, CAD tools, introduction to VHDL
- Section 3: Implementation Technology
  - Transistors operating as switches, NMOS and CMOS logic gates, ICs, PLAs, PALs, CPLDs, FPGAs, cell and gate arrays, voltage levels, noise margins, fan-in and fan-out, tri-state buffers
- Section 4: Optimized Implementation of Logic Functions
  - Karnaugh map, minimum-cost implementation techniques, multilevel synthesis, more VHDL

Course Outline (Cont'd)

- Section 5: Number Representation and Arithmetic Circuits
  - Representation of numbers, simple adder/subtractor circuits, carry-lookahead adder, more VHDL
- Section 6: Combinational-Circuit Building Blocks
  - Multiplexers, decoders, demultiplexers, encoders, more VHDL
- Section 7: Flip-Flops, Registers, Counters, and a Simple Processor
  - Latches, D flip-flop, T flip-flop, JK flip-flop, registers, shift registers, counters, bus structure, timing analysis, more VHDL
- Section 8: Synchronous Sequential Circuits
  - Moore/Mealy synchronous sequential circuits, state minimization, finite state machines, more VHDL

The lab component of this course is essential to understanding digital systems. There will be labs on basic digital logic, VHDL, and Altera ModelSim environment.

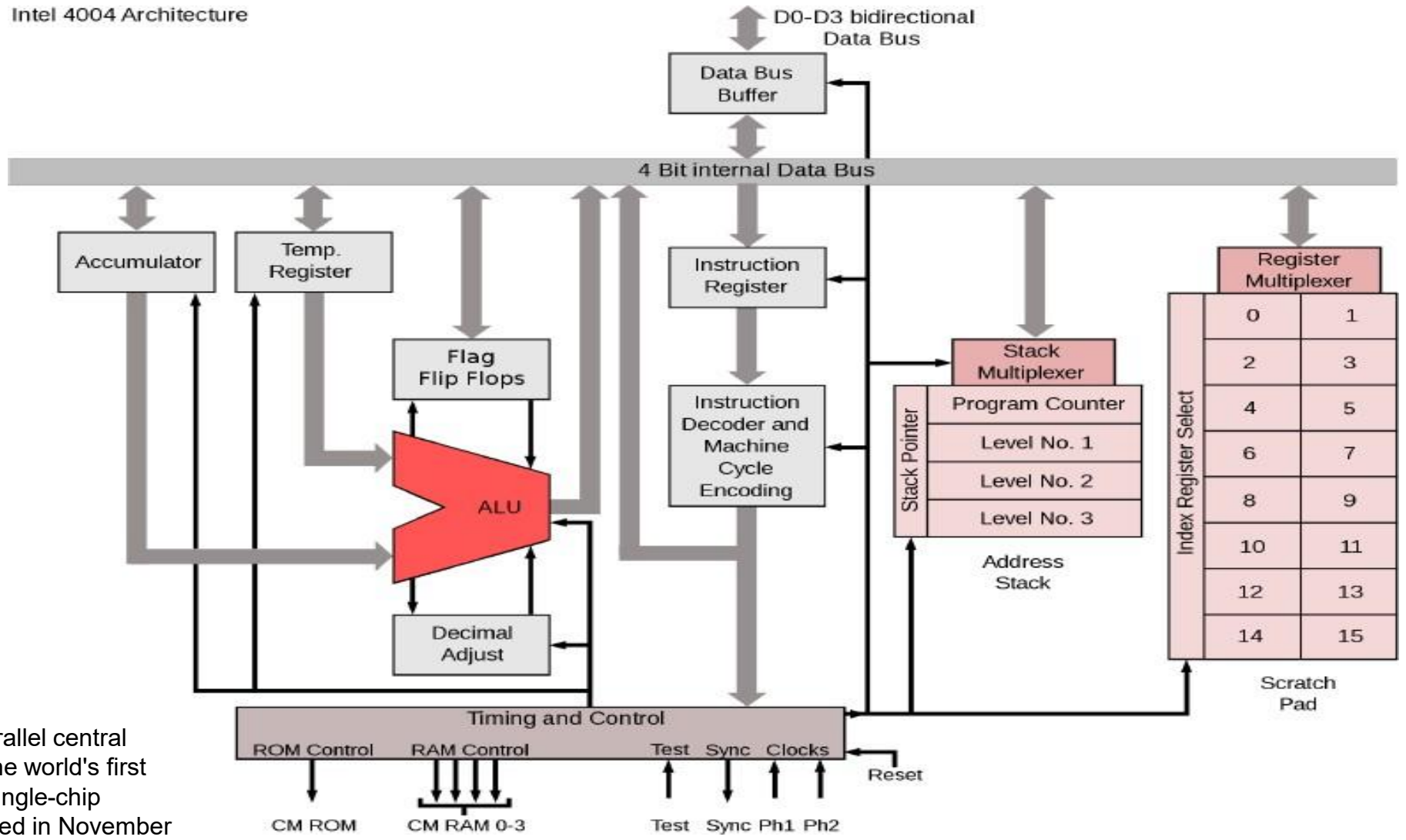
A little bit about me – Dr. Muhammad Saad Asad

- Joined Queen's in 2019
- Completed MASc (2019 - 2021) and PhD (2021 - 2025) programs at the department of Electrical and Computer Engineering.
- Specialization in nanophotonics and optics
- Research focus on metasurface and nanoscale multilayer thin film technology for applications including
  - Low-cost renewable heating
  - Nanoscale planar lens design
  - Dynamic polarization control
  - Optical beam steering

New Year New Experiences

- It's a first day for a lot of things
  - First second year digital course!
- ELEC 271 is fundamental, so you'll use this stuff throughout your career (**prerequisite to ELEC 274 and ELEC 371**)
  - First time thinking about this sort of subject
  - Can be difficult as it's the first time you're exposed to these kind of ideas
  - Allocate time to think about these ideas yourself and take the time to read other materials as well if it's not "clicking"
- Some of the material in this course you need to learn even if it's not on the exam
  - Important for your future careers, hard to test
  - Probably some of the most important material long-term

Common Theme in the Course



The Intel 4004 is a 4-bit parallel central processing unit (CPU). It is the world's first commercially available single-chip microprocessor and was released in November 1971.

ELEC 271

Digital Systems



Lecture Outline

- Digital Hardware
- Moore's Law, Technology Trends, Chip Types
- Layers of Abstraction
- Design Process
- Scope of this Course
- Design Flow for Logic Circuits
- Fixed Point Numbers and Positional Number Representation

Read Chapter 1 of the textbook.

Digital Hardware

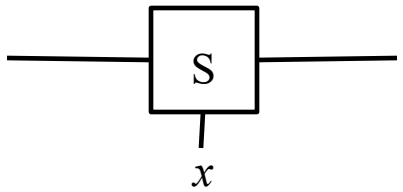
- An analog signal does not rely on discrete values, it can be an essentially infinite number of values (or levels) ex. A radio signal.

- A digital signal is one where there are agreed upon discrete values; for example, a **binary** system where levels are “1” or “0”. In practice an example is a voltage < 0.5V is considered “0”, while a voltage > 2.8V is considered a “1”.

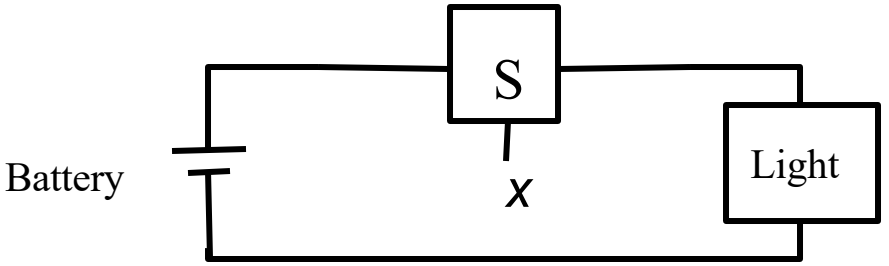
- A switch is a basic element in implementing a digital system.



Two states of a switch



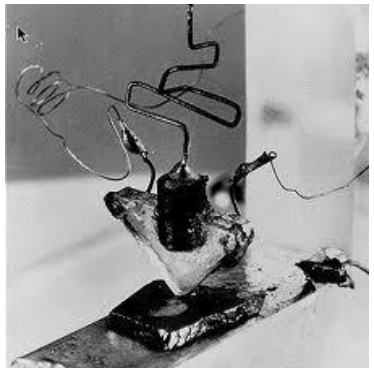
Symbol for a switch

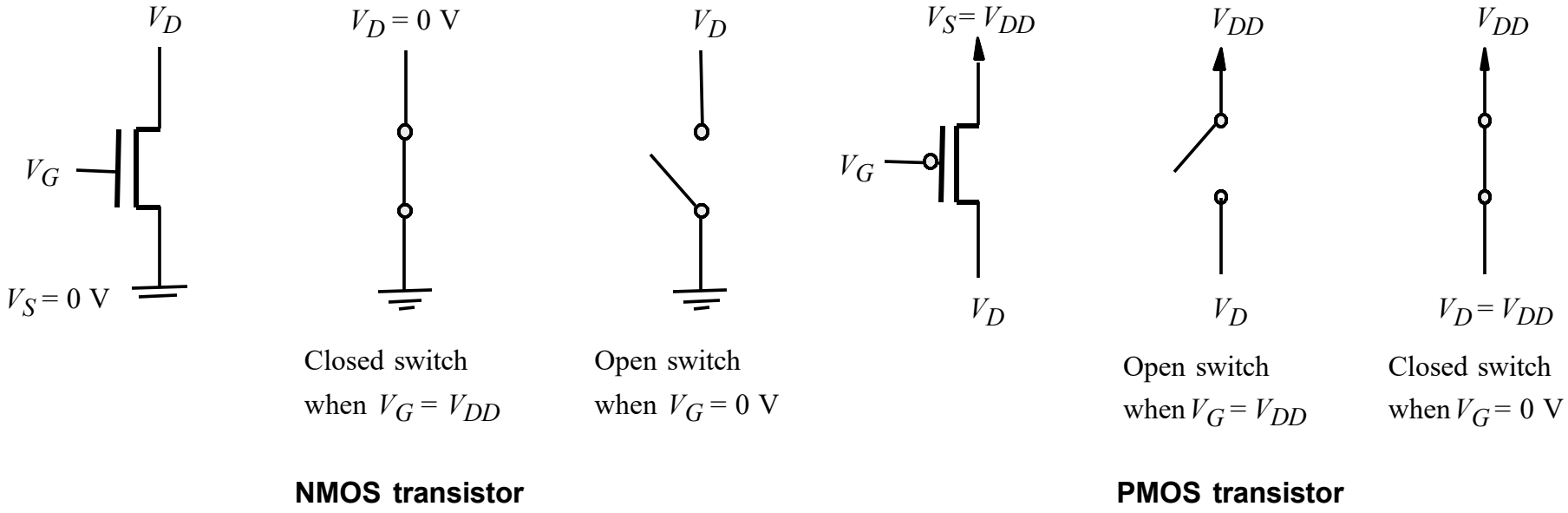


Simple connection to a battery

Digital Hardware (Cont'd)

- Early digital systems used electromechanical relays (vacuum tubes). Modern digital systems use **transistors** as the switching element.
- Up to 1960's, discrete transistors (invented in 1947 at Bell Laboratories) were used to implement digital systems.
- There are two major "types" of transistors, NMOS and PMOS

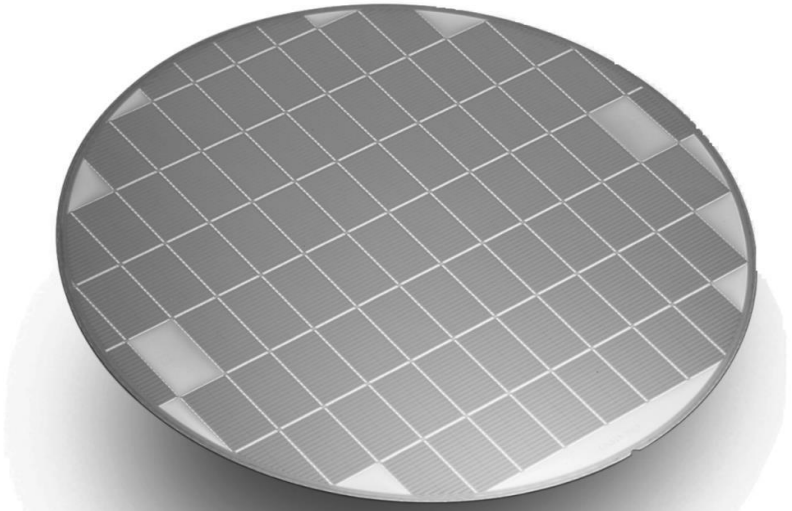




Simple use of NMOS and PMOS transistors in digital systems

Digital Hardware (Cont'd)

- Until the 1960s logic circuits were constructed with bulky components, such as transistors and resistors that came as individual parts.
- Integrated Circuit (IC): IC technology was born in early 1960's (in fact, in 1958 by Texas Instruments), and the first microprocessor on a die was invented by Intel in 1970.
- The advent of integrated circuits made it possible to place a number of transistors, and thus an entire circuit, on a single chip.



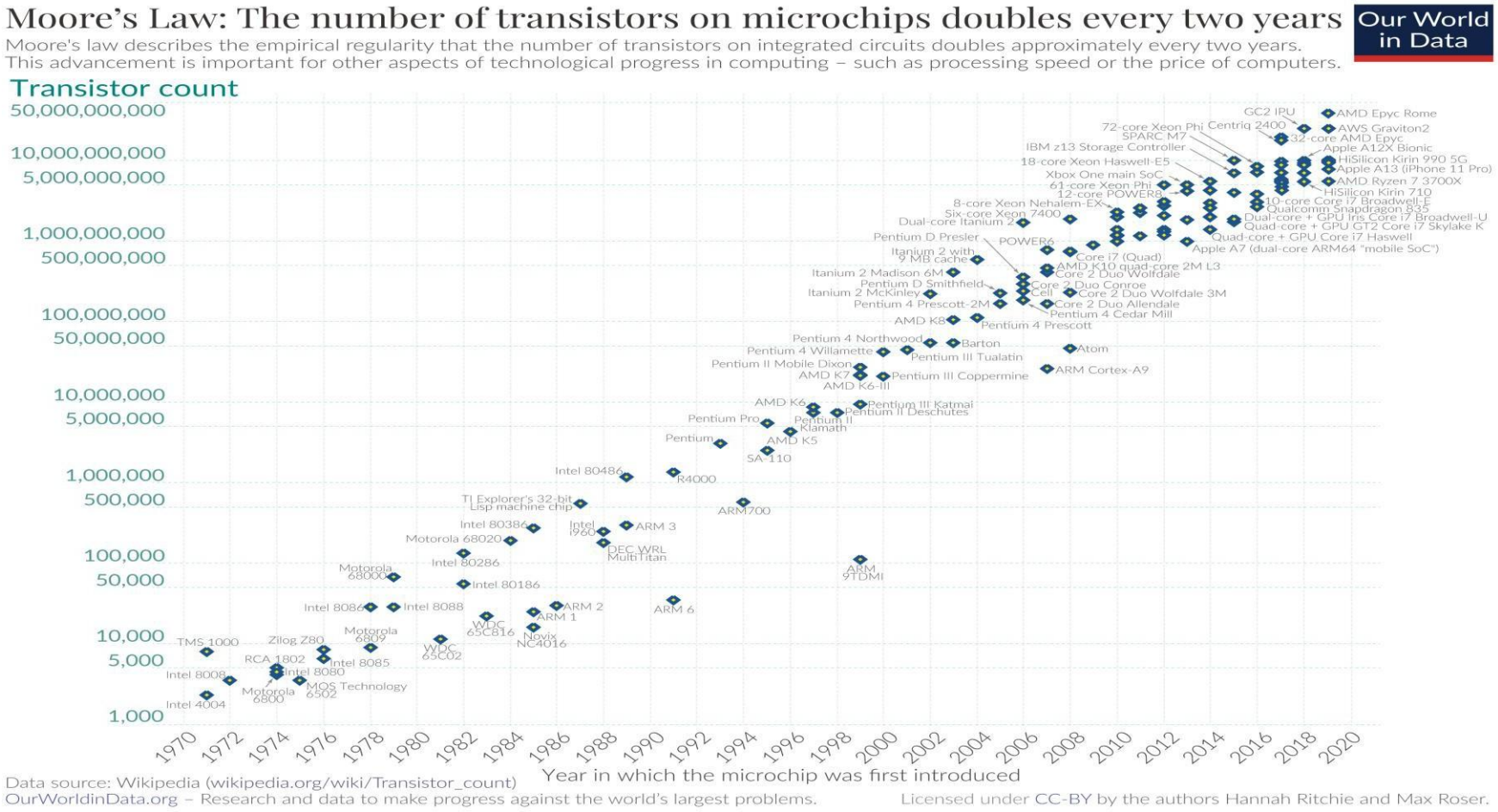
A silicon wafer

Zooming in a CPU

- https://www.youtube.com/watch?v=xZIZ3LWyhvc

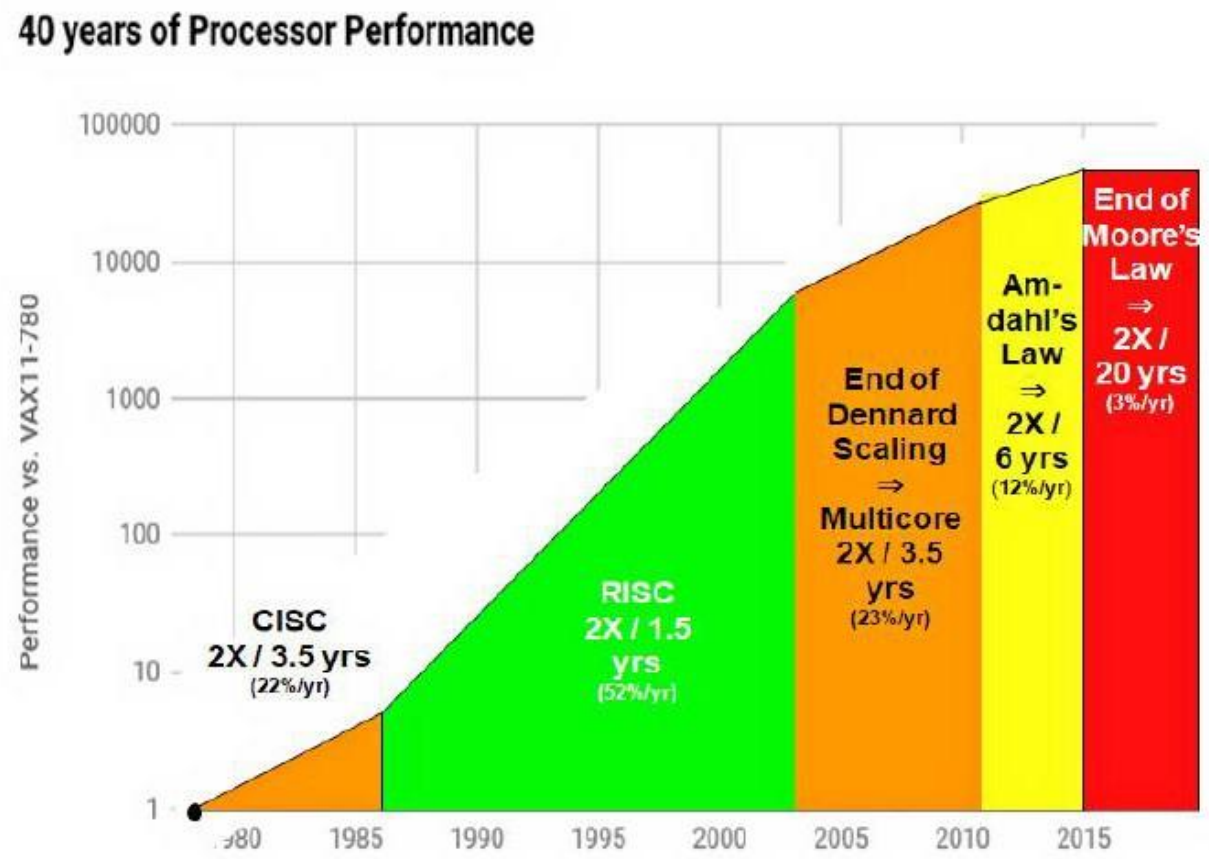
Moore's Law

- Moore's Law – chip density: an empirical observation that the **number of transistors on a chip is doubling every 18 months**. Intel expects new transistor tech to lead to trillion transistor chips by 2030!



Technology Trends

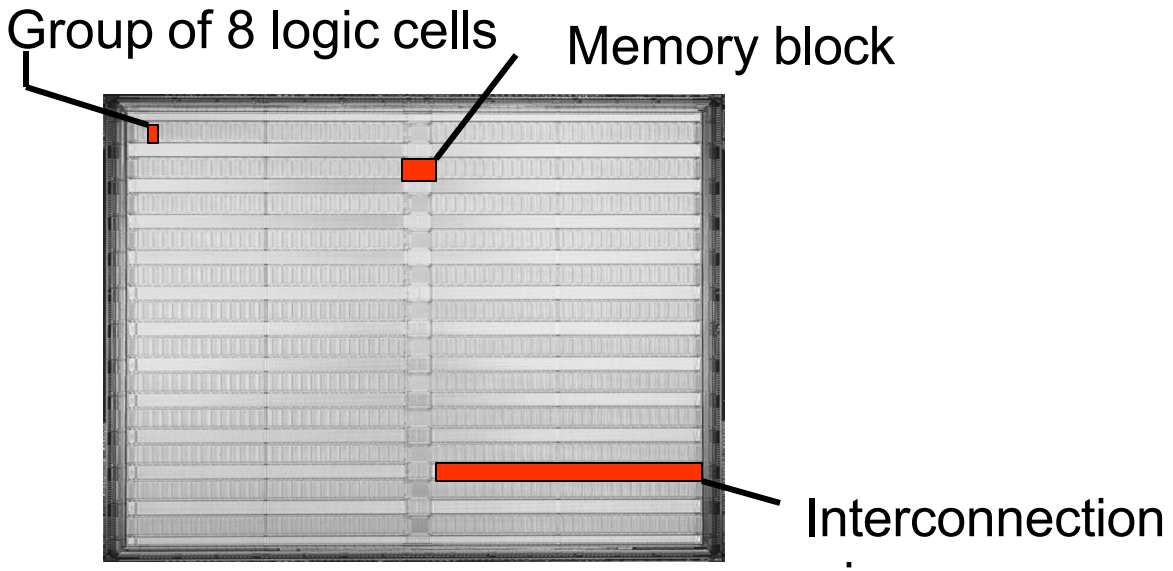
- Everyone predicts the end of Moore's Law, and yet we keep going.
- Patterson even thinks so, he's one of the inventors of RISC!



Source: John Hennessy and David Patterson, Computer Architecture: A Quantitative Approach, 6/e. 2018

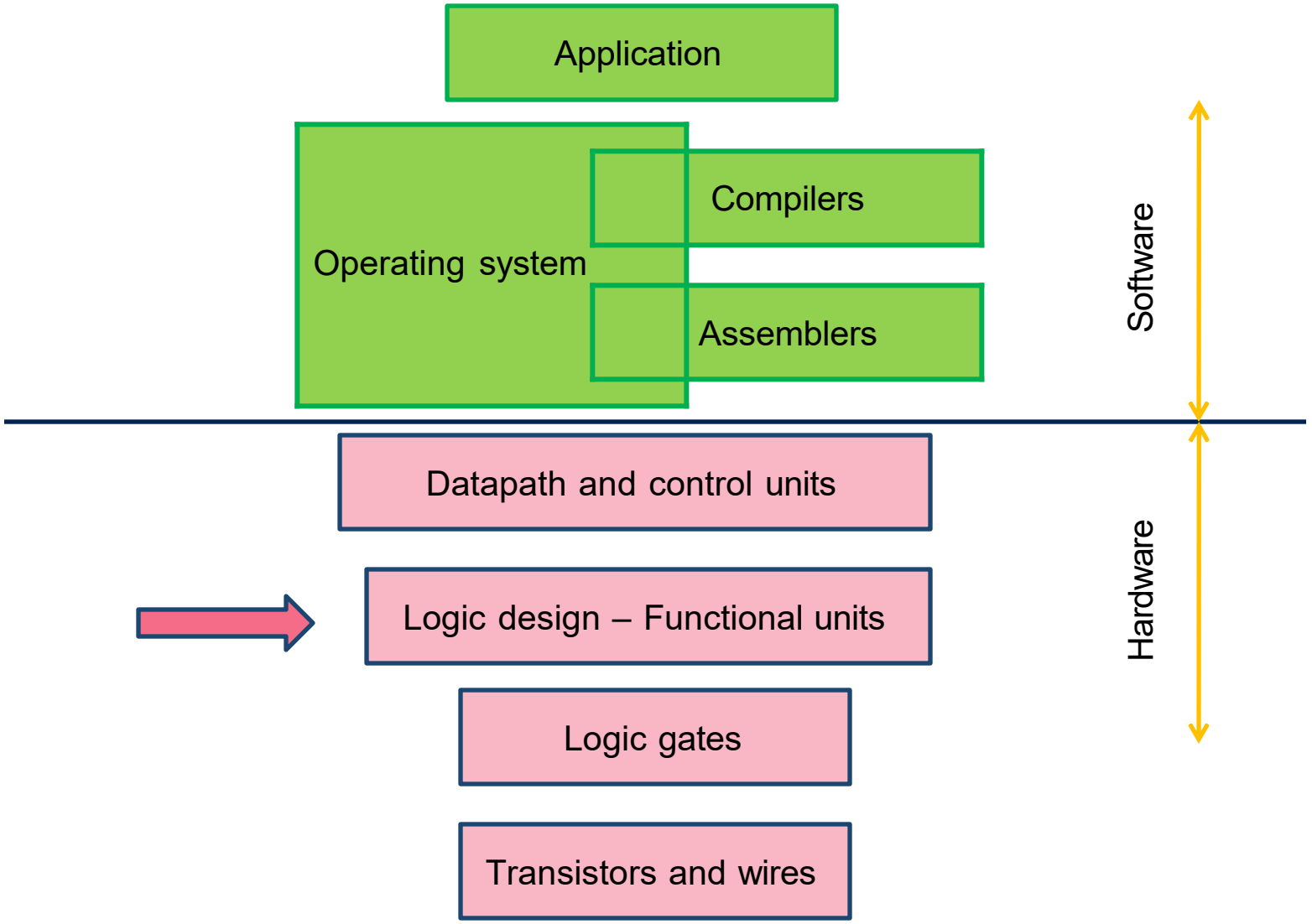
Chip Types

- Standard chips: 74 series, 40/45 series (small and fixed functionality. Conform to an agreed upon standard in terms of functionality and physical configuration)
- Programmable logic devices: Programmable Logic Devices (PLDs), complex PLDs (CPLDs), field-programmable gate arrays (FPGA) – consist of programmable switches. In contrast to standard chips that have fixed functionality, it is possible to construct chips that contain circuitry which can be configured by the user to implement a wide range of different logic circuits. Programmable switches consume valuable space which may limit speed of operation.
- Custom-designed chips: Application-specific ICs (ASICs), Gate Array, Standard Cells – design optimized for a particular task leading to better performance; high time-to-market.



A field-programmable gate array chip

Layers of Abstraction



Design Process

- Logic design: given a desired system specification, design a digital system that meets some criteria, such as speed, cost, power, etc.
  - Choose the logic components
  - Choose implementation technology
- Goals:
  - Design should be correct and meet the specification.
  - It should be cost-effective; costs can be classified as time-to-market, power dissipation, area, etc.
  - Design should be easy to upgrade or enhance.
  - Design should be reliable, and work without error.
    - For these, designers may need to maximize speed, optimize logic, minimize power, lower cost of manufacturing, lower time-to-market, use CAD tools, etc.

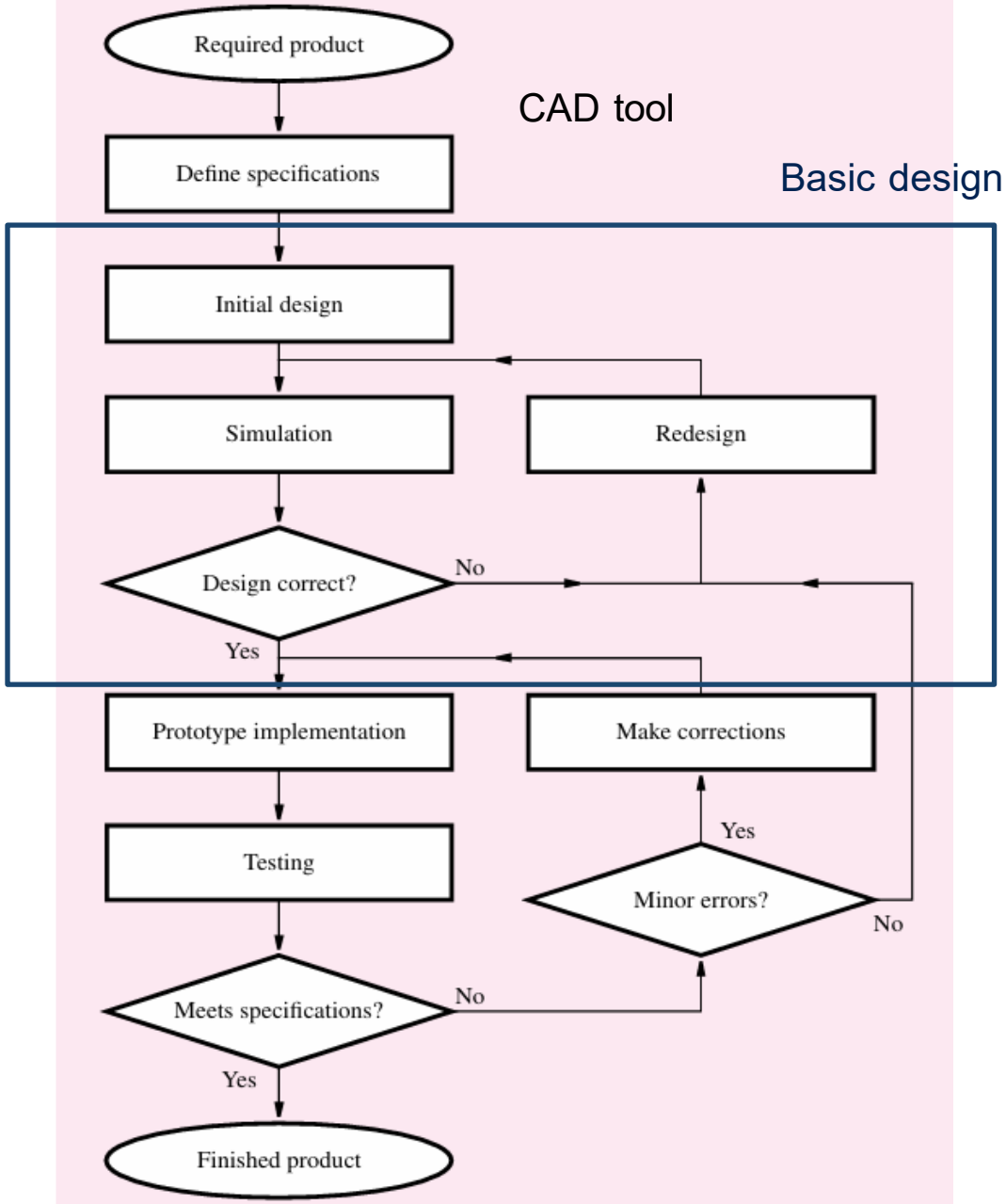
Design Process (Cont'd)

- Significant changes such as access to VLSI and ULSI technology, cheaper and faster time-to-market designs are happening right now in the hardware design phase.

- This is only possible because of significant progress in:
  - Utility and acceptability of CAD environment over traditional manual methods
  - Simulation techniques to verify the large, complex logic design
  - Automatic synthesis methods
  - Using programmable chips such as FPGA

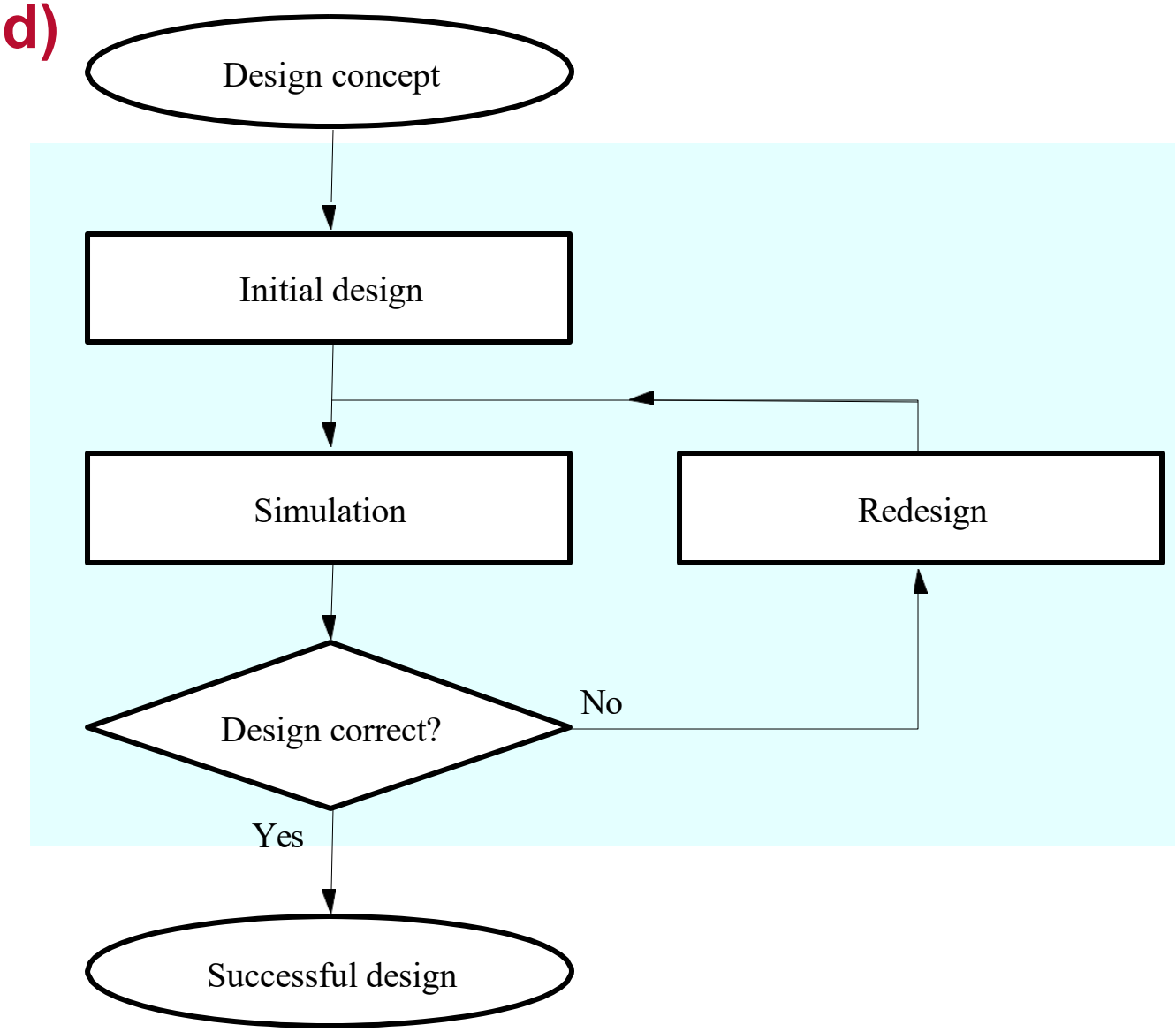
Design Process (Cont'd)

- A typical development process

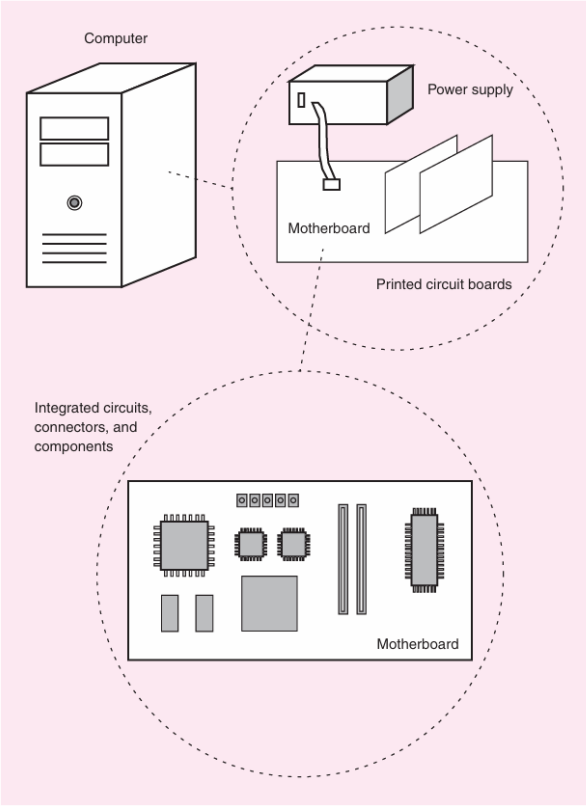


Design Process (Cont'd)

Note that while CAD tools provide efficiency and do many tasks automatically, the designer is still responsible for designing the digital system based on the knowledge of the underlying theory of logic design. It is also the designer who is familiar with the nature of systems, and chooses the different components.

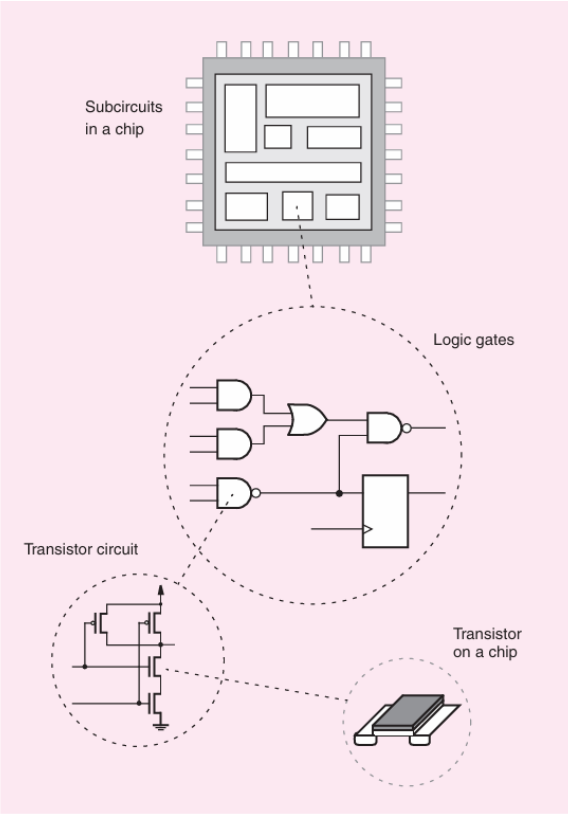


Scope of this course



A digital hardware system (Part a)

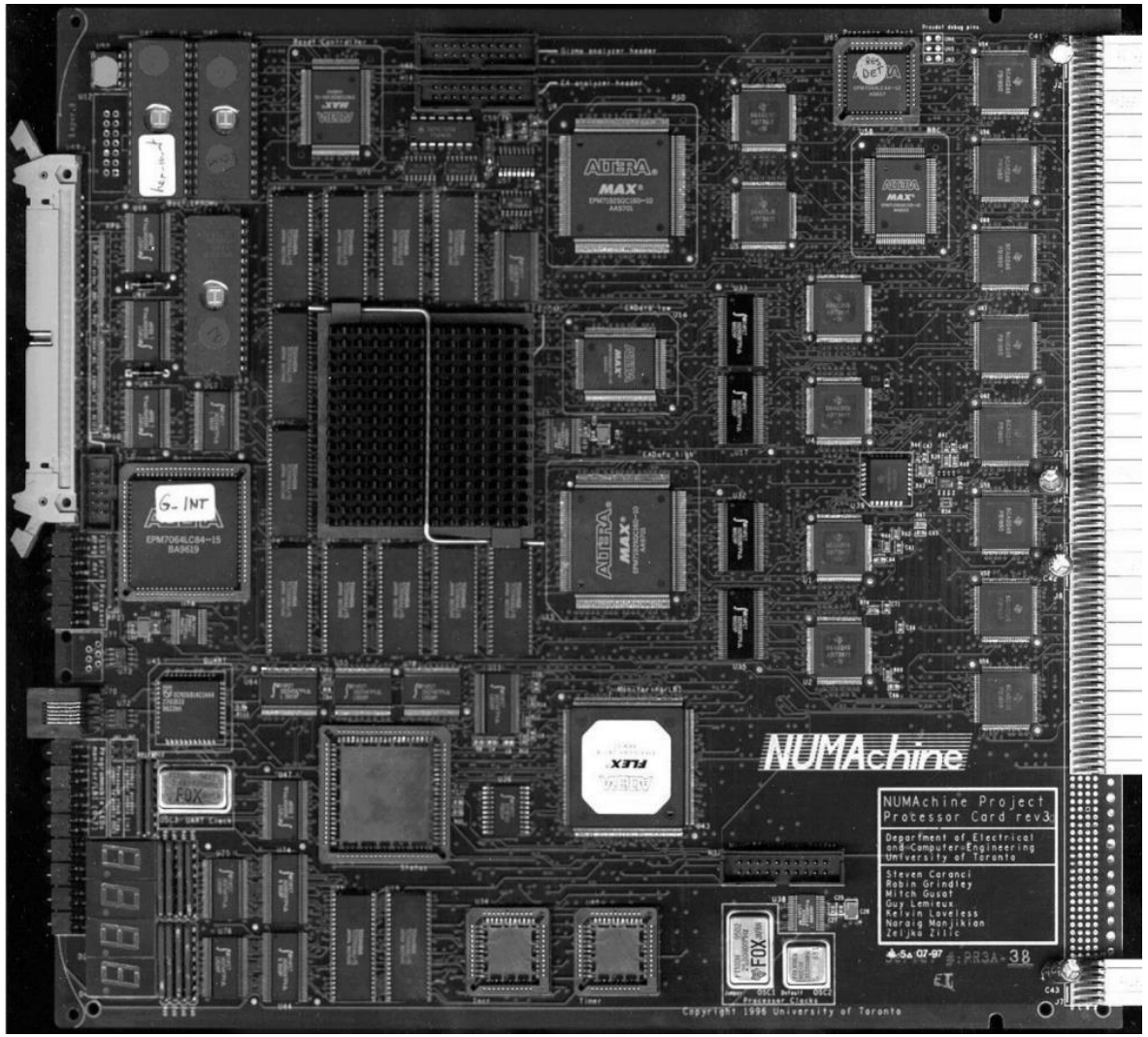
Consider the structure of a typical computer, as illustrated in Figure 1.4a. The computer case houses a number of printed circuit boards **(PCBs), a power supply, and storage units**, like a hard disk. Each unit is plugged into a main PCB, called the motherboard. The motherboard holds several integrated circuit chips.



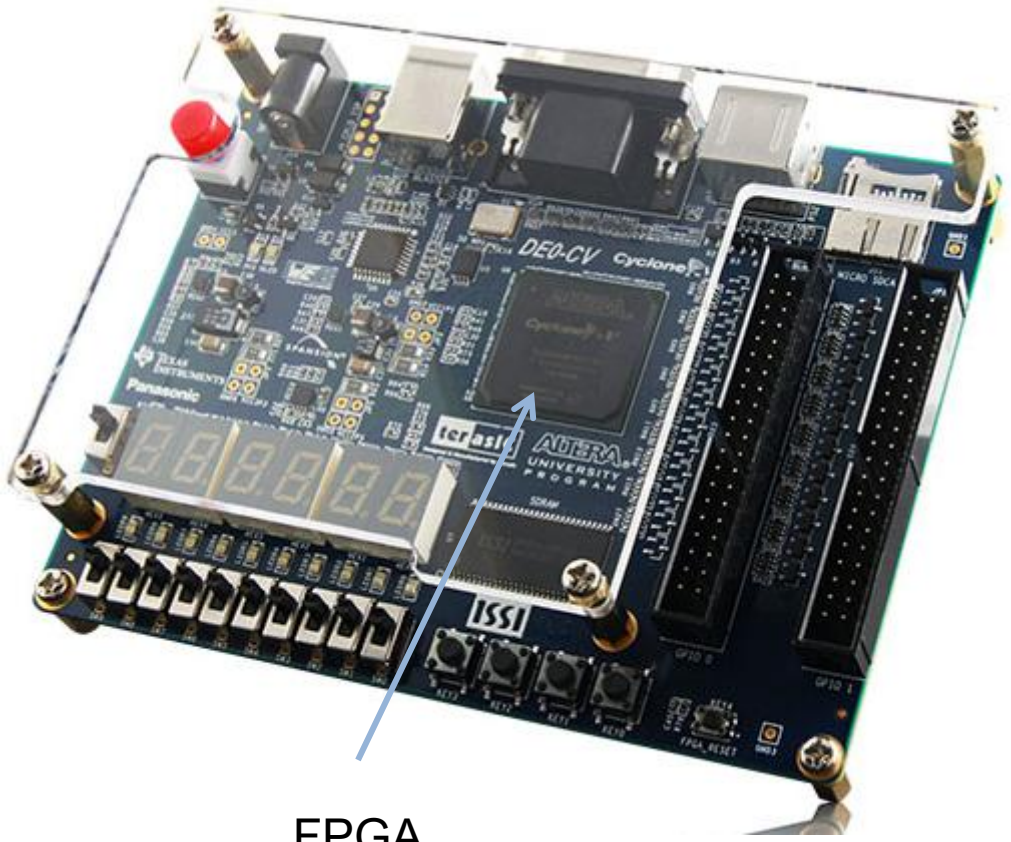
A digital hardware system (Part b)

Figure 1.4b illustrates the **structure of an integrated circuit chip**. The chip comprises a number of **logic circuits**. As shown in the middle of the figure, a logic circuit comprises a network of connected **logic gates**. Each logic gate performs a very simple function, and more complex operations are realized by connecting gates together. Logic gates are built with transistors

A Printed Circuit Board



A PCB

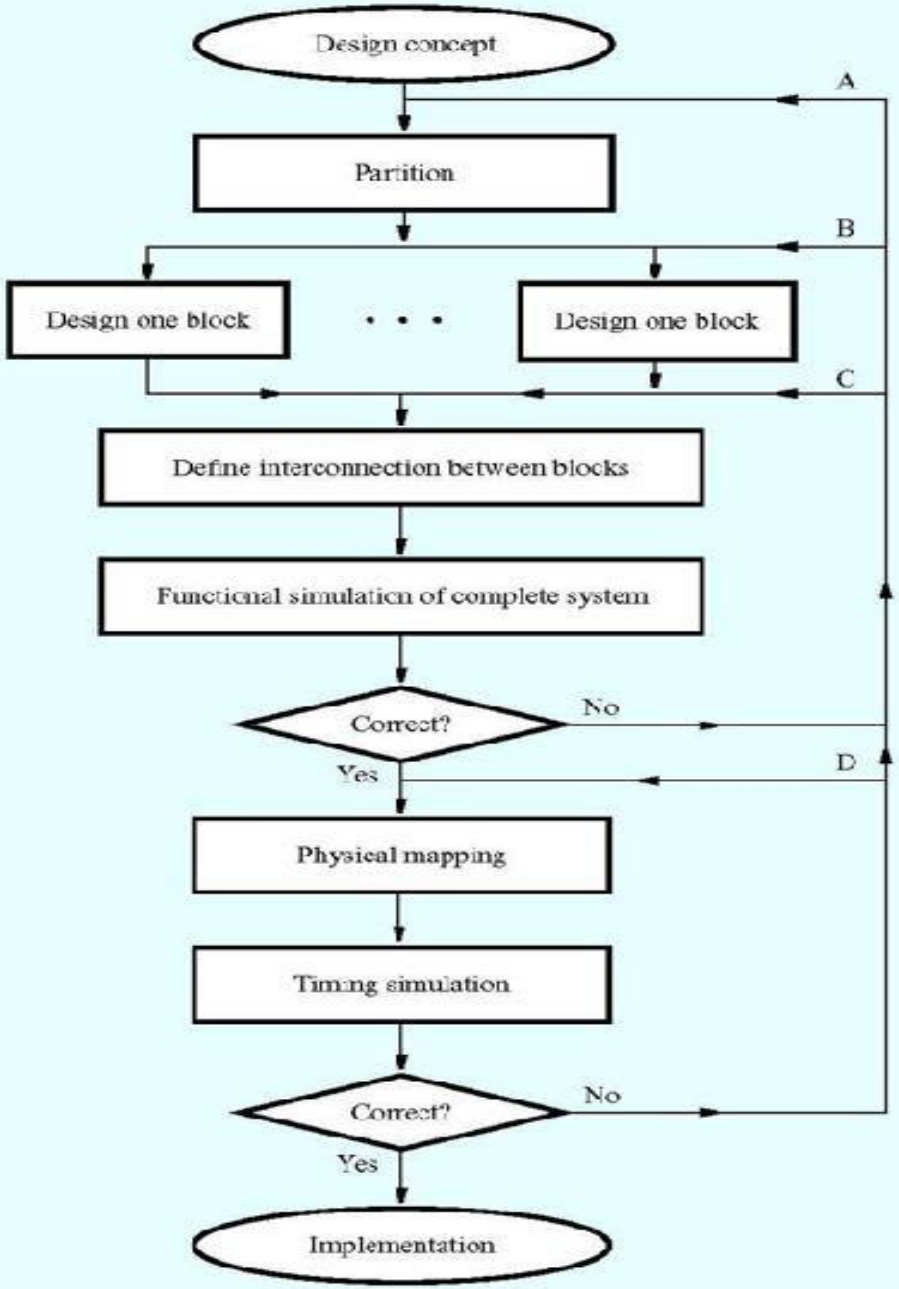


FPGA

PCB with FPGA (Altera Cyclone V)

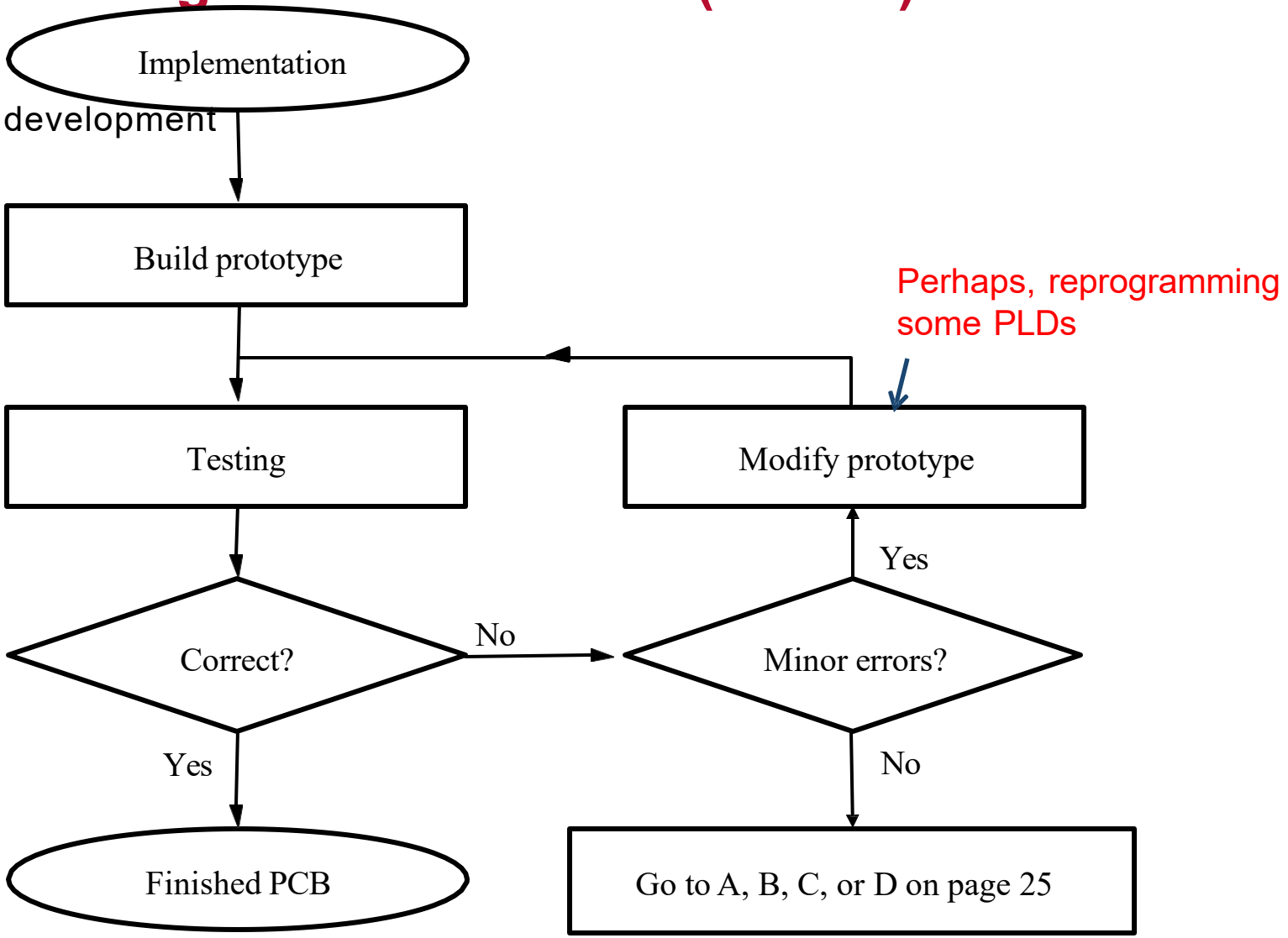
Design Flow for Logic Circuit Boards

A: Partitioning problem
B: Poor block design
C: Incorrect connections
D: Wire impact on signal delay



Design Flow for Logic Circuit Boards (Cont'd)

- Completion of PCB development



The Basics of Binary Numbers

Fixed Point Numbers and Positional Number Representation

- The base or radix of a number system defines the range of possible values that a digit may have: 0 - 9 for decimal; 0, 1 for binary.
- In any fixed point number system, each fixed point number has exactly the same number of digits, and the fixed point is always in the same place. For instance, 11.10, 01.10 in base 2, or 5.11, 0.34 in base 10.
- Fixed-point representation is a method of storing real numbers in a computer system where the position of the decimal or binary point is fixed and remains in the same position. This method allocates a specific, unchanging number of bits for the integer part and the fractional part.

- **The general form for determining the decimal (base 10) value of a number in base *k* is given by:**

Positional number representation (polynomial method)

(Used to convert from base k to decimal value)

$$Value = \sum_{i=-m}^{n-1} b_i . k^i$$

n is the number of digits before decimal point.
m is the number of digits after decimal point.
b_i is digit at position i.
k The base of the original number (e.g., k=2 for binary, k=16 for hexadecimal).

Fixed Point Numbers and Positional Number Representation

Positional number representation (polynomial method)

$$Value = \sum_{i=-m}^{n-1} b_i . k^i$$

n is the number of digits before decimal point.
m is the number of digits after decimal point.
b_i is digit at position i.
k The base of the original number (e.g., k=2 for binary, k=16 for hexadecimal).

- Example: Binary to decimal conversion
  - Using *polynomial method*, find the decimal equivalent of the number $(1010.01)_2$

$(1010.01)_2 = 1 \times 2^{-2} + 0 \times 2^{-1} + 0 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3$

$= ¼ + 0 + 0 + 2 + 0 + 8 = (10.25)_{10}$

Note: If there is **no decimal point**, the sum goes from $i = 0 \; to \; i = n - 1$ as we will see in the next few slides

Binary numbers

- In decimal system, the number consists of digits that have 10 possible values from 0-9, and each digit represents a multiple of a power of 10.
- For example, the number 8547 represents: $8 \times 10^3 + 5 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$
- We do not normally write the powers of 10 with the number, because they are implied by the positions of the digits.
- In general, a decimal integer is expressed by an n-tuple comprising n decimal digits

$$D = d_{n-1}d_{n-2} \cdots d_1 d_0$$

- Which represents the value:

$$V(D) = d_{n-1} \times 10^{n-1} + d_{n-2} \times 10^{n-2} + \cdots + d_1 \times 10^1 + d_0 \times 10^0$$

- This is referred to as the ***positional number representation***.
- Because the digits have **10 possible values** and **each digit is weighted as a power of 10**, we say that decimal numbers are **base-10 numbers**

Binary numbers contd.

- Decimal numbers are familiar, convenient, and easy to understand.
- However, since digital circuits represent information using only the values 0 and 1, it is not practical to have digits that can assume ten values.
- In these circuits it is more appropriate to use the binary, or base-2, system which has only the digits 0 and 1.
- Each binary digit is called a ***bit***.
- In the binary number system, the same positional number representation is used so that

$$B = b_{n-1}b_{n-2}\cdots b_1 b_0$$

- represents an integer that has the value

$$\begin{aligned} V(B) &= b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \\ &= \sum_{i=0}^{n-1} b_i \times 2^i \end{aligned}$$

Binary numbers contd. binary to decimal conversion

For example, the binary number 1101 represents the value

$$V = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Because a particular digit pattern has different meanings for different bases, we will indicate the base as a subscript when there is potential for confusion.

1101 as a base-2 number is written as $(1101)_2$

Evaluating the preceding expression for V gives V =8+4+1=13. Hence

$$(1101)_2 = (13)_{10}$$

Binary numbers contd. binary to decimal conversion

Some examples of numbers in decimal (from 0 to 15) and in binary form

Table 1.1 Numbers in decimal and binary.

Decimal representation	Binary representation
00	0000
01	0001
02	0010
03	0011
04	0100
05	0101
06	0110
07	0111
08	1000
09	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

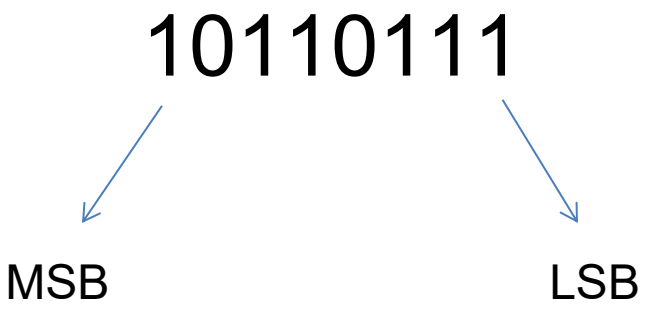
An example of a larger number is:

$$(10110111)_2 = (183)_{10}$$

In general, using n bits allows representation of positive integers in the range 0 to 2^{n-1}

Binary numbers contd.

- In a binary number the right-most bit is usually referred to as the **least-significant bit (LSB)**.
- The left-most bit, which has the highest power of 2 associated with it, is called the **most-significant bit (MSB)**.
- In digital systems it is often convenient to consider several bits together as a group. A group of four bits is called a nibble, and a group of eight bits is called a byte.



Base Conversion: Remainder Method

So far, we have seen how to convert from **base k to decimal (base 10)** using polynomial method.

To convert from **decimal to base k** we use remainder and multiplication method.

Conversion from decimal to binary example shown on the right.

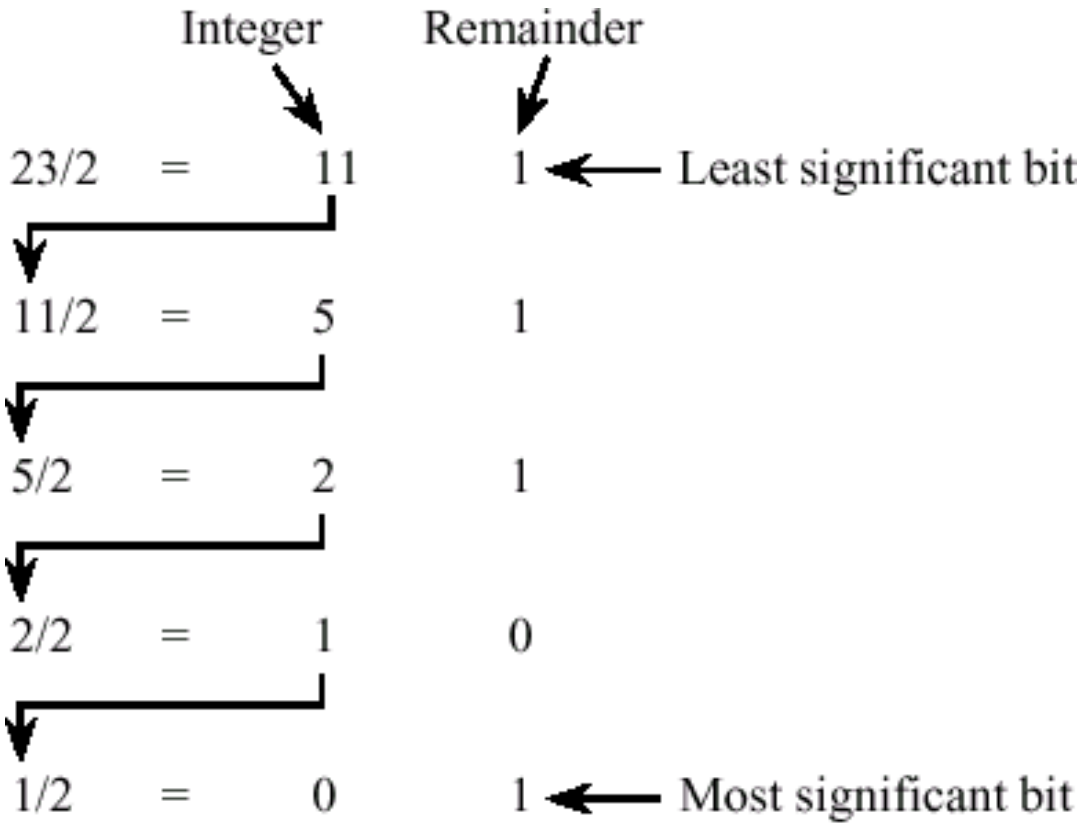
Convert $(857)_{10}$

			Remainder	
$857 \div 2$	=	428	1	LSB
$428 \div 2$	=	214	0	
$214 \div 2$	=	107	0	
$107 \div 2$	=	53	1	
$53 \div 2$	=	26	1	
$26 \div 2$	=	13	0	
$13 \div 2$	=	6	1	
$6 \div 2$	=	3	0	
$3 \div 2$	=	1	1	
$1 \div 2$	=	0	1	MSB

Result is $(1101011001)_2$

Base Conversion: Remainder Method

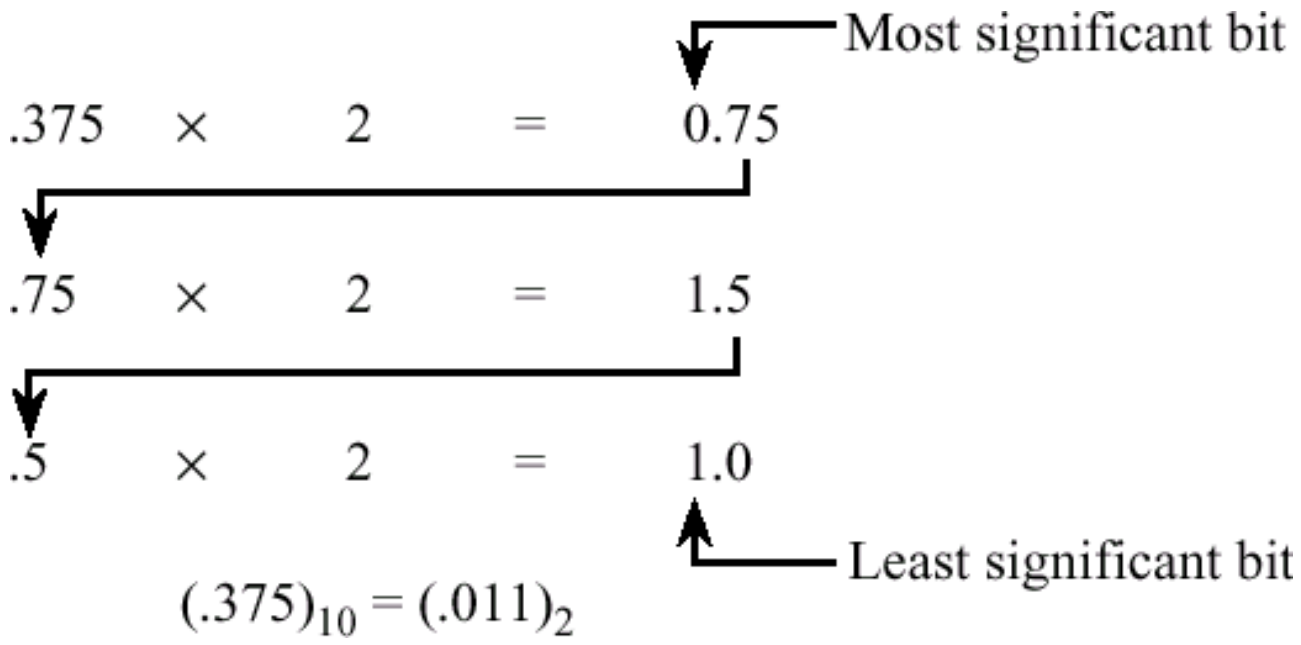
- Example: Decimal to binary conversion
  - Convert 23.375_{10} to base 2.
  - Start with the integer part.



$$(23)_{10} = (10111)_2$$

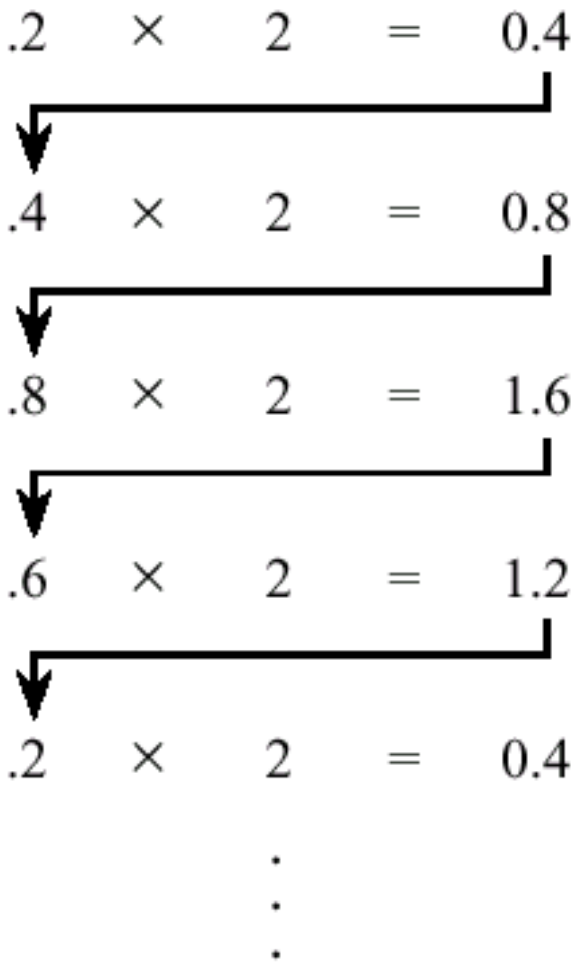
Base Conversion: Multiplication Method

- Now, convert the fraction part.



Non-terminating Fractions

- Not all terminating base 10 fractions have a terminating base 2 form.
- Example: convert $.2_{10}$ to base 2.



Base 2, 8, 10, and 16 Number systems

- Most computers use base 2 for internal representation and arithmetic. However, it needs more digits to represent a number. Thus, it is easy to make errors when writing them.

  ➔ Use base 8 (*octal*), or base 16 (*hexadecimal*).

Binary (base 2)	Octal (base 8)	Decimal (base 10)	Hexadecimal (base 16)
0	0	0	0
1	1	1	1
10	2	2	2
11	3	3	3
100	4	4	4
101	5	5	5
110	6	6	6
111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F

Base 2, 4, 8, and 16 Conversions

- Example:

  $1011_2 = (10_2)\,(11_2) = (2_4)\,(3_4) = 23_4$

  $101010_2 = (101_2)\,(010_2) = (5_8)\,(2_8) = 52_8$

  $10110_2 = (010_2)\,(110_2) = (2_8)\,(6_8) = 26_8$

  $01101101_2 = (0110_2)\,(1101_2) = (6_{16})\,(D_{16}) = 6D_{16}$

  $10110110_2 = (1011_2)\,(0110_2) = (B_{16})\,(6_{16}) = B6_{16}$

  $11001_2 = (0001_2)(1001_2) = (1_{16})\,(9_{16}) = 19_{16}$

  $231_4 = (2_4)\,(3_4)\,(1_4) = (10_2)\,(11_2)\,(01_2) = 101101_2$

  $537_8 = (5_8)\,(3_8)\,(7_8) = (101_2)\,(011_2)\,(111_2) = 101011111_2$

  $A6E_{16} = (A_{16})\,(6_{16})\,(E_{16}) = (1010_2)\,(0110_2)\,(1110_2) = 101001101110_2$

- For an unnatural conversion, **convert first to base 10 as an intermediate step (using polynomial method), then convert from base 10 to the target base (using remainder and multiplication methods).**

Base 2, 4, 8, and 16 Conversions

- Hexadecimal is base 16
- Since $16 = 2^4$, we need a group of 4 bits to convert from binary to hexadecimal.
- Similarly,
  - for base 8, since $8 = 2^3$, we need groups of 3 bits
  - for base 4, since $4 = 2^2$, we need groups of 2 bits

- **Example, Express $(10110110)_2$ in base 16 and base 8**
  - $(10110110)_2 = (1011)_2(0110)_2 = (B)_{16}(6)_{16} = (B6)_{16}$
  - $(10110110)_2 = (010)_2(110)_2(110)_2 = (2)_8(2)_8(6)_8 = (226)_8$

Need to add an extra 0 at the start to equally split is groups of 3 bits

- **Example, Express $(10101)_2$ in base 16**
  - $(10101)_2 = (0001)_2(0101)_2 = (1)_{16}(5)_{16} = (15)_{16}$

- **Example convert $(527)_8$ to base 16**
- First convert to binary $(527)_8 = (101010111)_2 = (0001)_2(0101)_2(0111)_2$
  $= (157)_{16}$

Binary (base 2)	Octal (base 8)	Decimal (base 10)	Hexadecimal (base 16)
0	0	0	0
1	1	1	1
10	2	2	2
11	3	3	3
100	4	4	4
101	5	5	5
110	6	6	6
111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F



Queen's
UNIVERSITY